

# PROJECT SUBMISSION REPORT

## Production-Ready DevOps Pipeline for AI Microservice

### 1. Project Metadata & Code Layout

GitHub Repository URL: <https://github.com/bhumikasharna020/ai-microservice>

AWS EKS Cluster Name: ai-microservice-eks

Kubernetes Namespace: ai-microservice

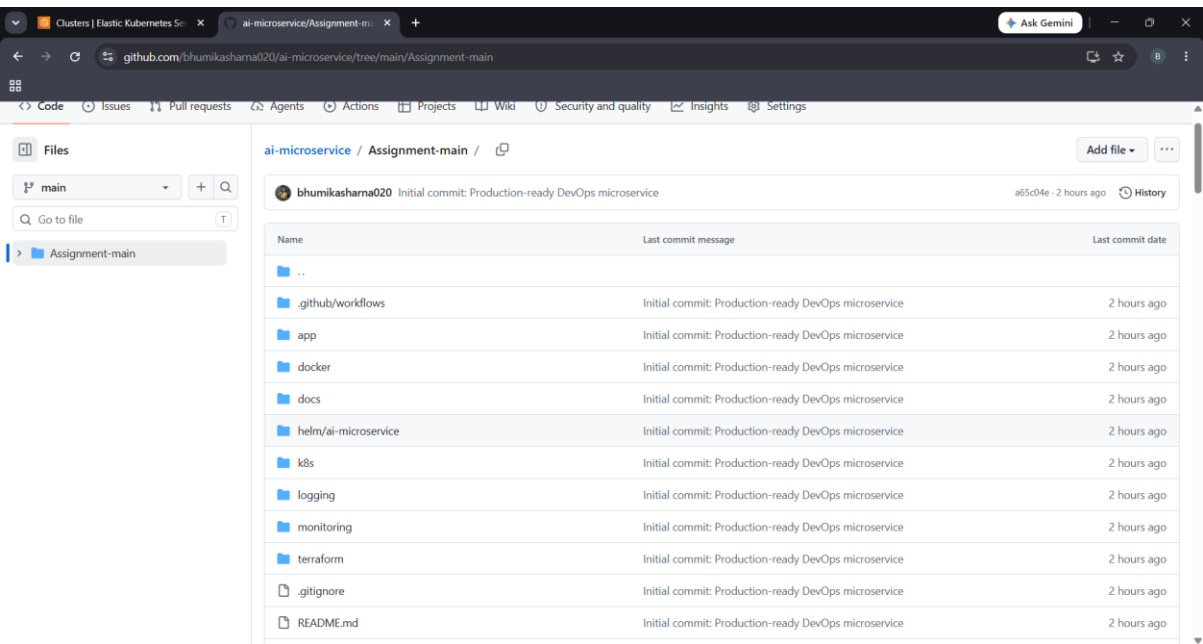


Figure 1.1: GitHub Repository Files & Directory Layout

### 2. Local Setup & Docker Containerization

We configured a multi-stage Docker build running under a non-root user (appuser UID 1000) for security hardening. The application and database run successfully locally.

#### 2.1 Docker Compose Local Startup

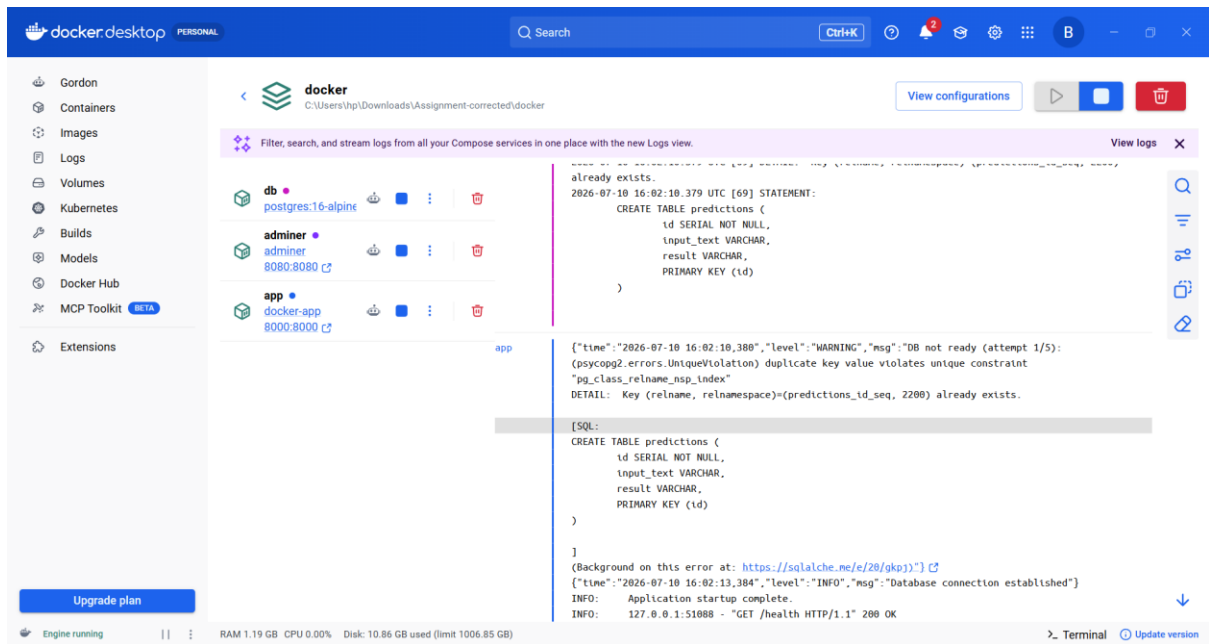


Figure 2.1: Docker Compose Local Startup Logs

## 2.2 Swagger API Documentation UI

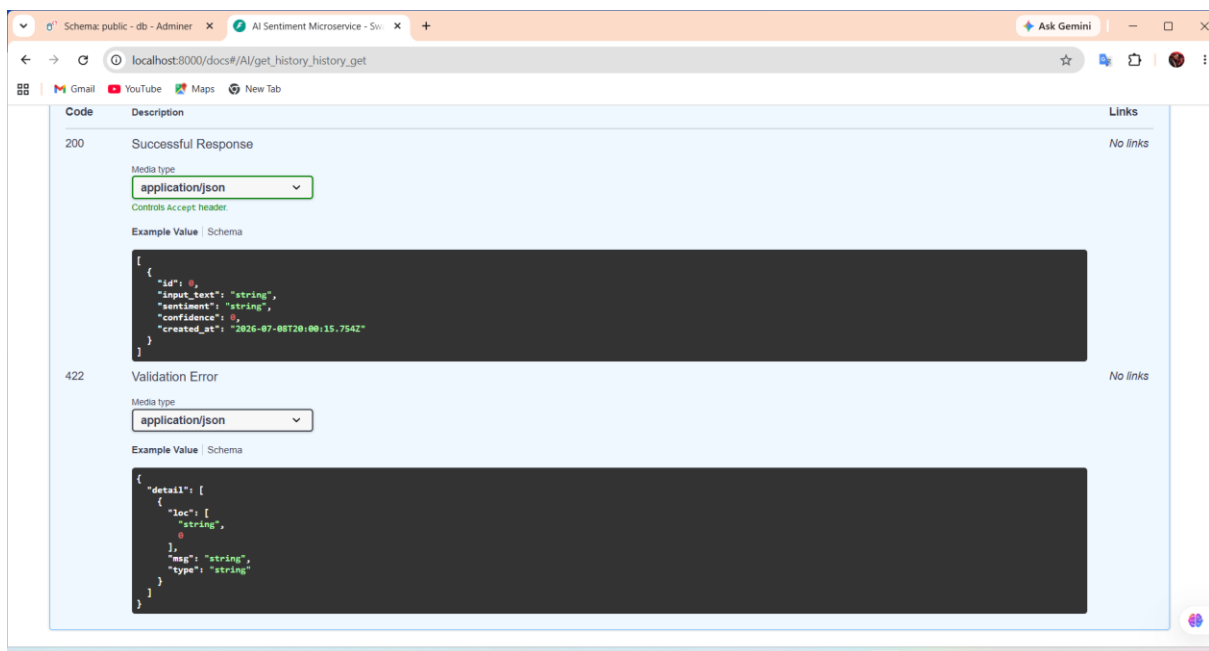


Figure 2.2: FastAPI Swagger Documentation Panel

## 2.3 PostgreSQL Database Storage Verification

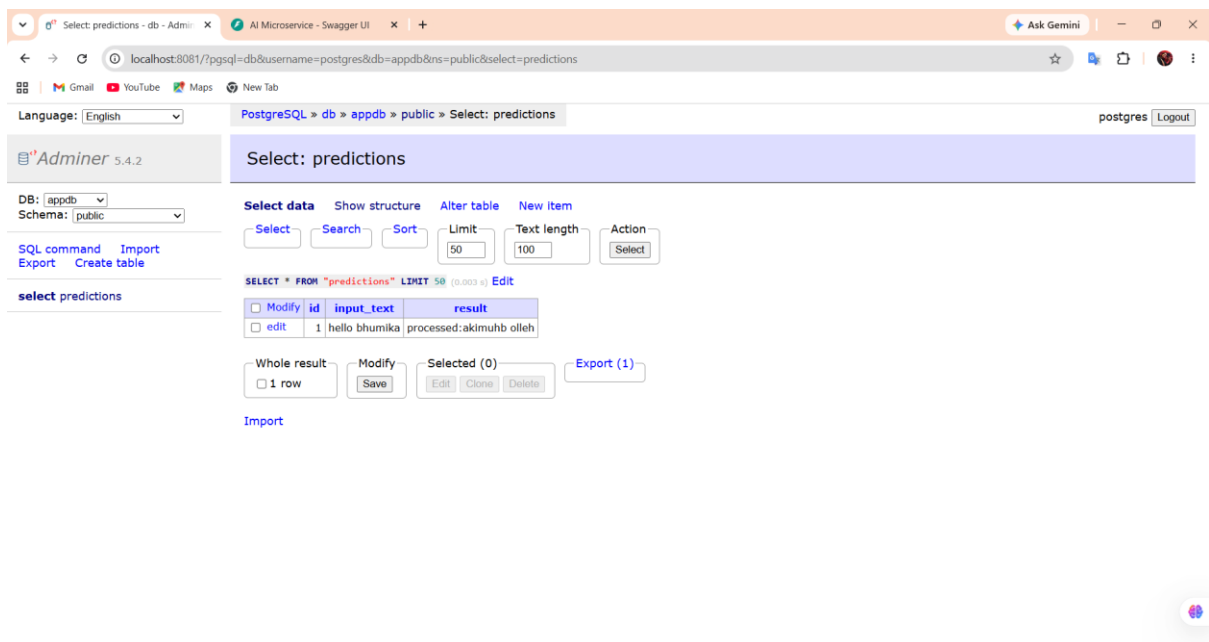


Figure 2.3: Adminer predictions Database Table Output showing saved text

## 2.4 Docker Hub Registry Push Verification

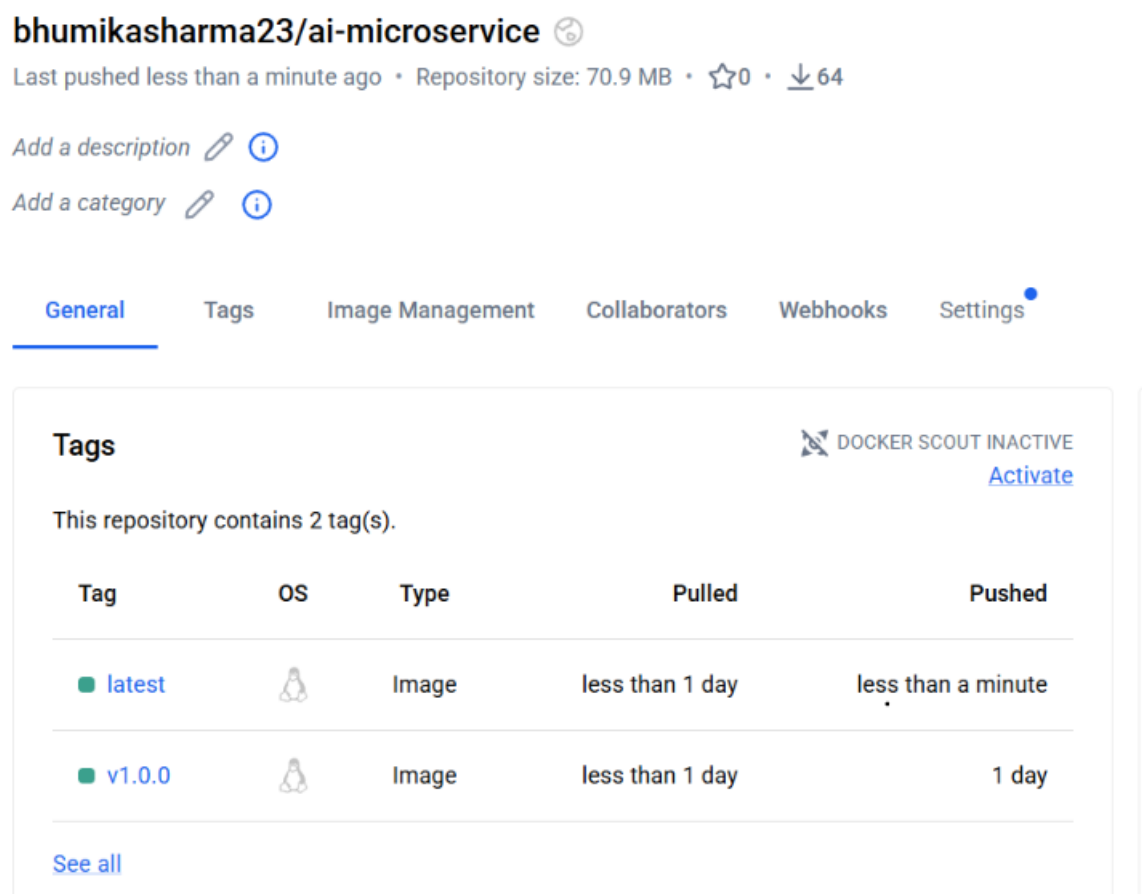


Figure 2.4: Pushed Image confirmation tags on Docker Hub Registry

## 3. Cloud Infrastructure & AWS EKS Provisioning

Automated cloud provisioning scripts deploy VPC, public/private subnets, NAT Gateway, EKS Cluster, and EC2 node groups.

### 3.1 AWS EKS Cluster Active Status

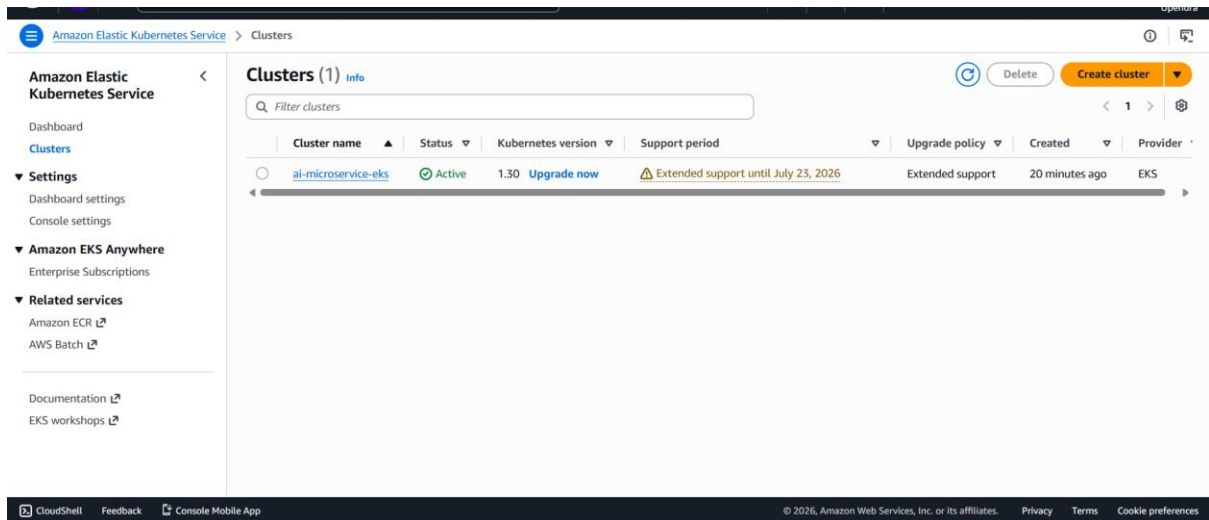


Figure 3.1: AWS Console active EKS Cluster Status

### 3.2 AWS EKS Cluster Overview Details

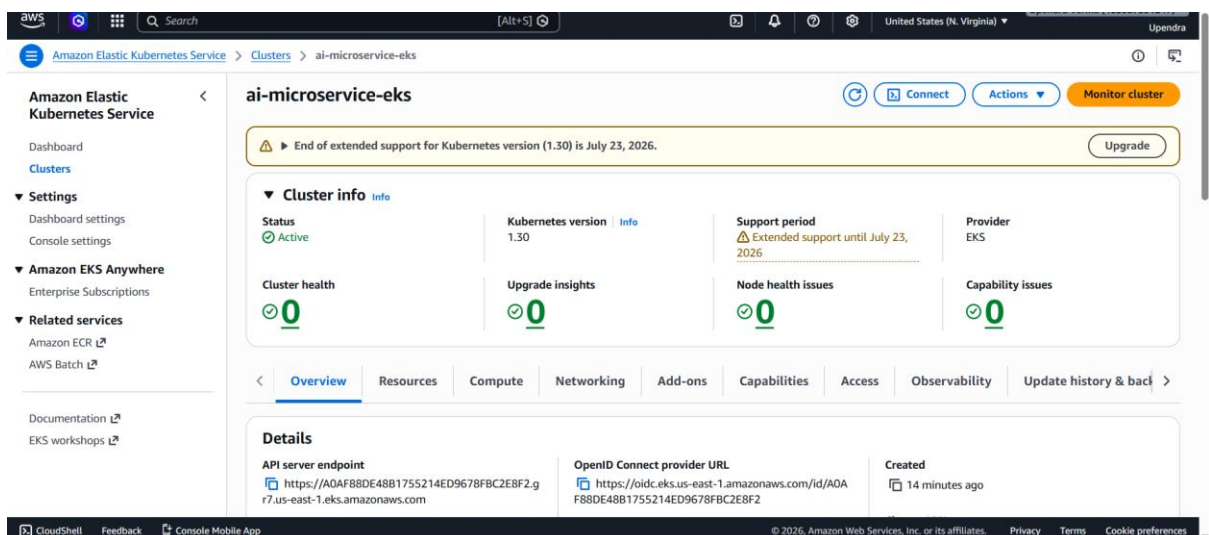
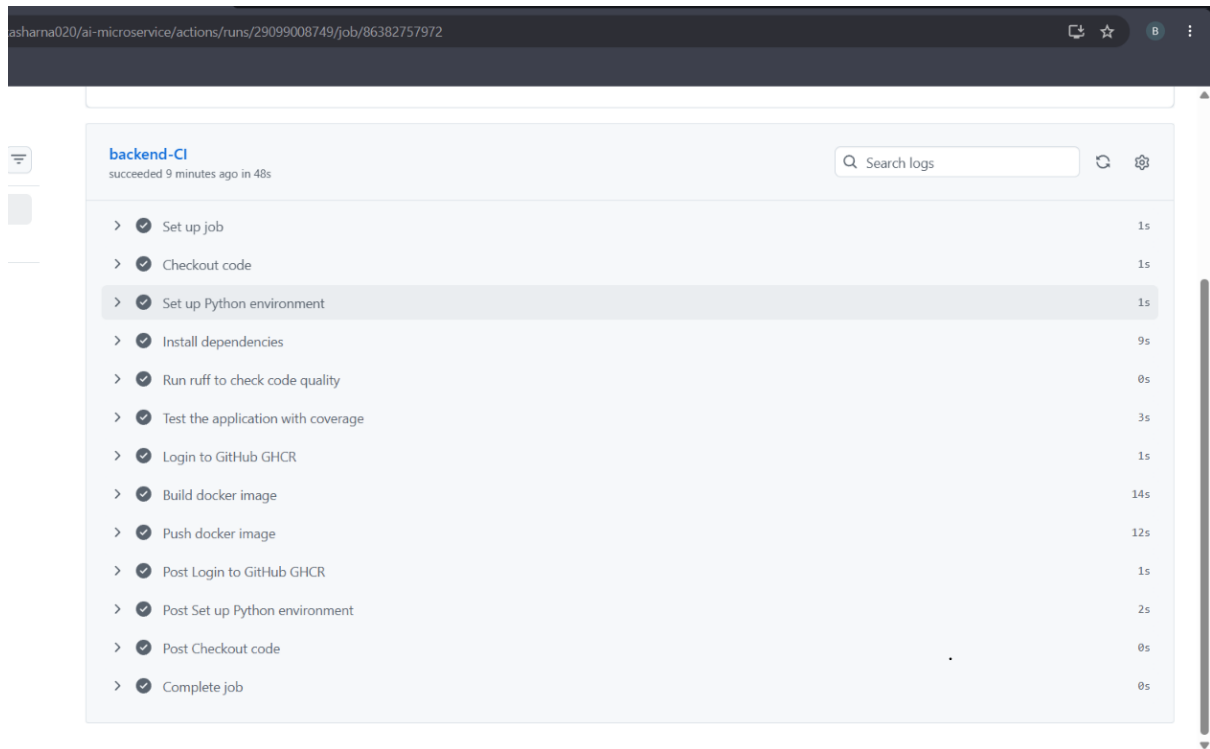


Figure 3.2: EKS details with Endpoint and OpenID Connect URLs

## 4. CI/CD Pipeline & Vulnerability Scan

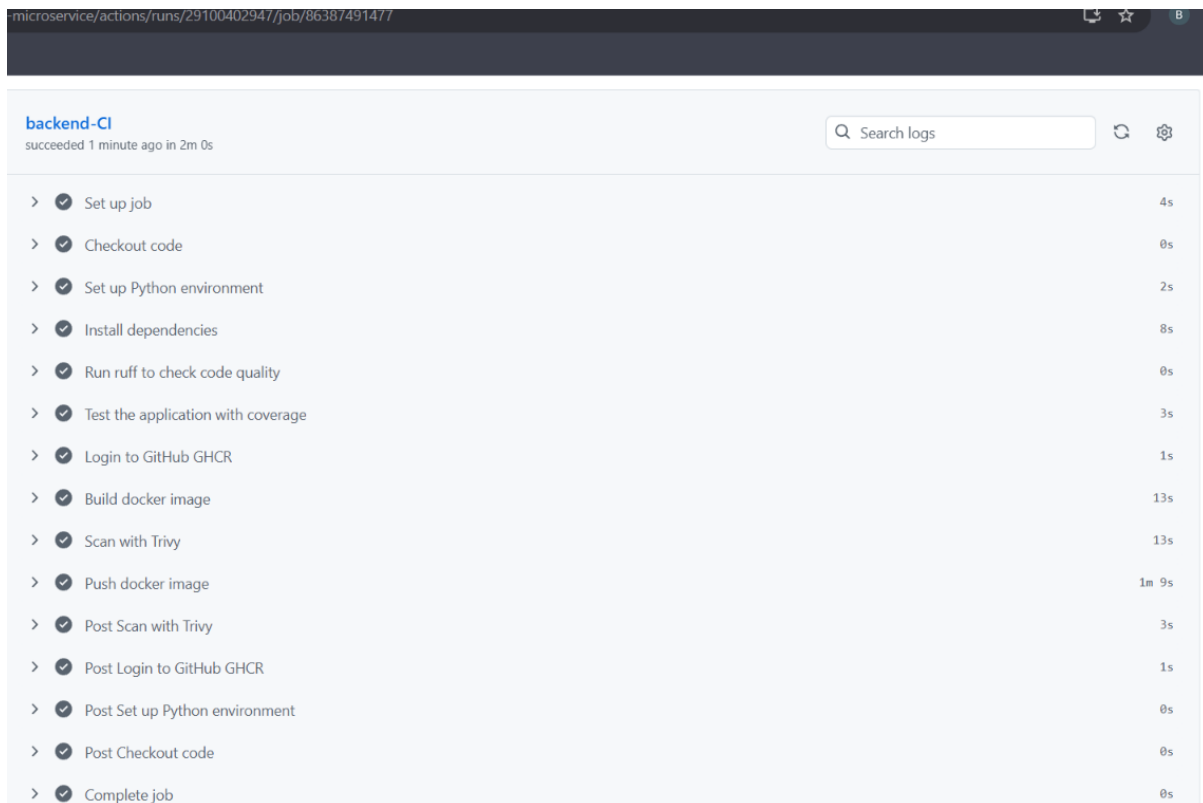
Our pipeline validates code quality using Ruff linter, runs pytest coverage, scans container with Trivy, and pushes the package to GHCR.

## 4.1 GitHub Actions Green CI Logs



**Figure 4.1: Succeeded GitHub Actions CI Pipeline job logs**

## 4.2 Trivy Container Vulnerability Scan Logs



**Figure 4.2: Trivy vulnerability report output showing 0 vulnerabilities**

## 5. Kubernetes Security & Hardening Manifests

We configured strict memory/CPU resource constraints, container probes, RBAC roles, and Network Policies.

### 5.1 RBAC ServiceAccount & Role Definition

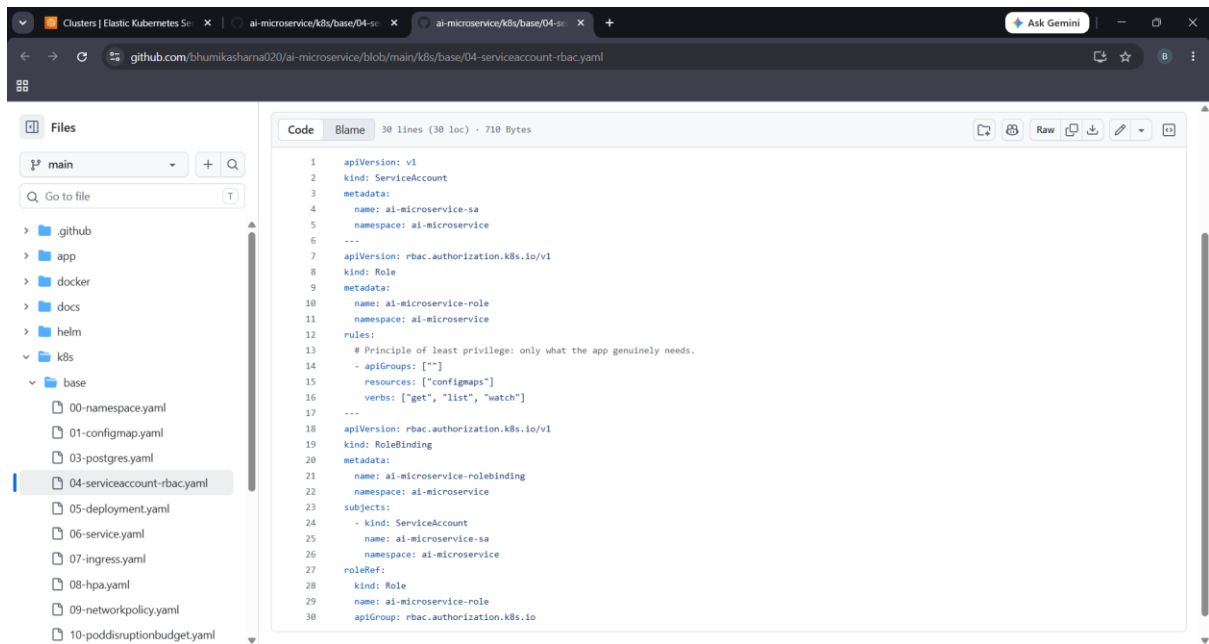


Figure 5.1: ServiceAccount and Roles configurations

## 5.2 Resource Limits & Health Probes

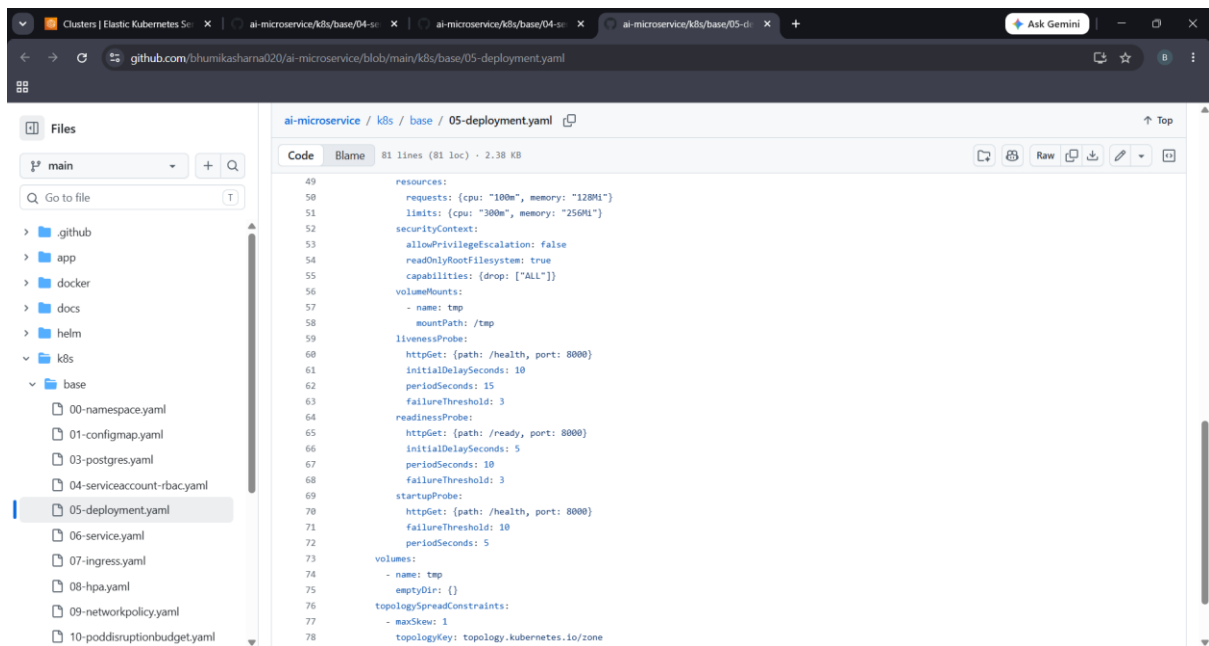
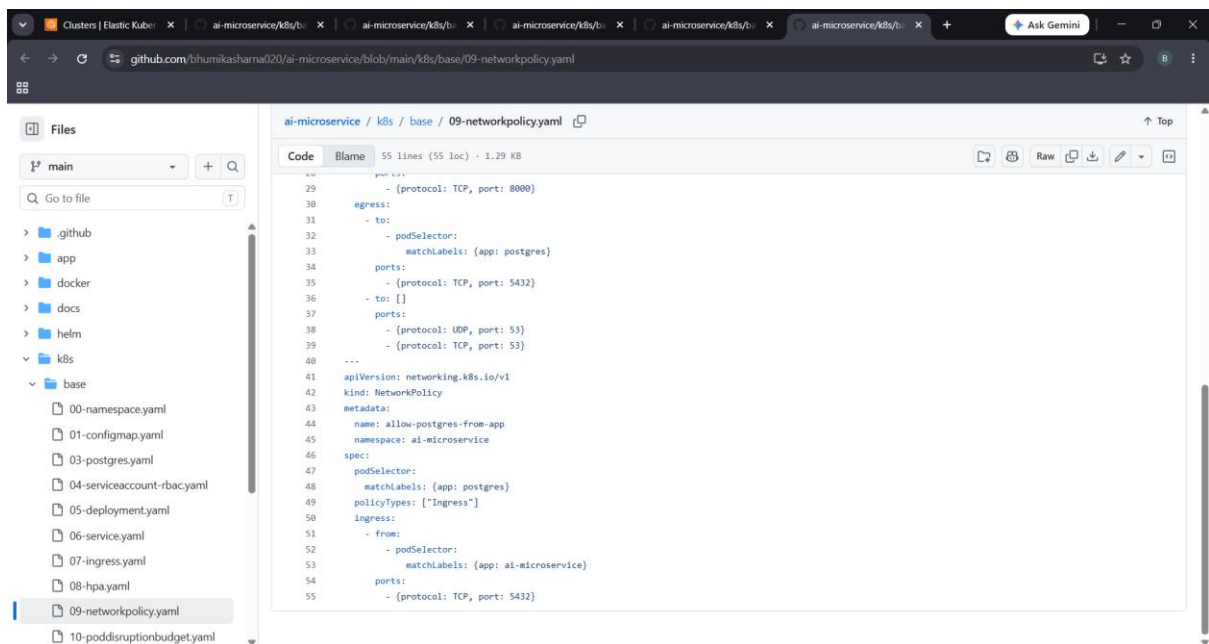
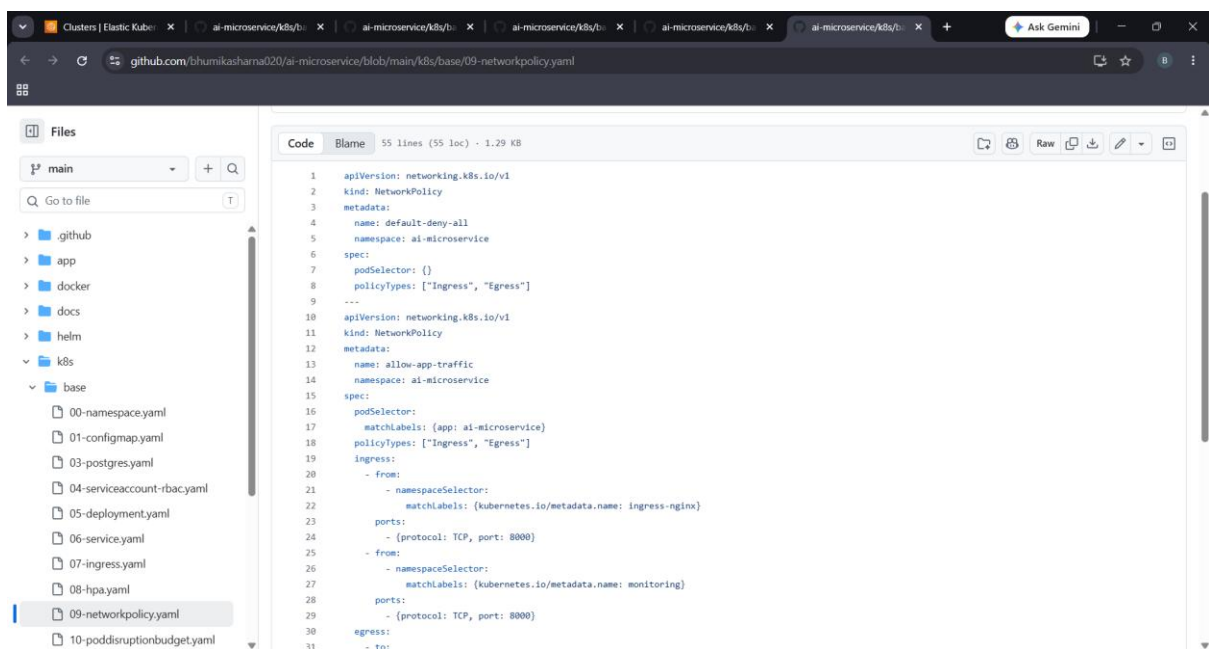


Figure 5.2: CPU/Memory requests vs limits and startup/readiness/liveness checks

## 5.3 Network Policy Rules



**Figure 5.3: Strict Network Policy block rules**



**Figure 5.4: PostgreSQL Ingress Network Policy rules**

## 6. Centralized Logging & Monitoring Dashboards

Centralized telemetry dashboards monitor cluster behavior using Prometheus metrics, Grafana charts, and Loki log streams.

### 6.1 Grafana Monitoring Dashboard

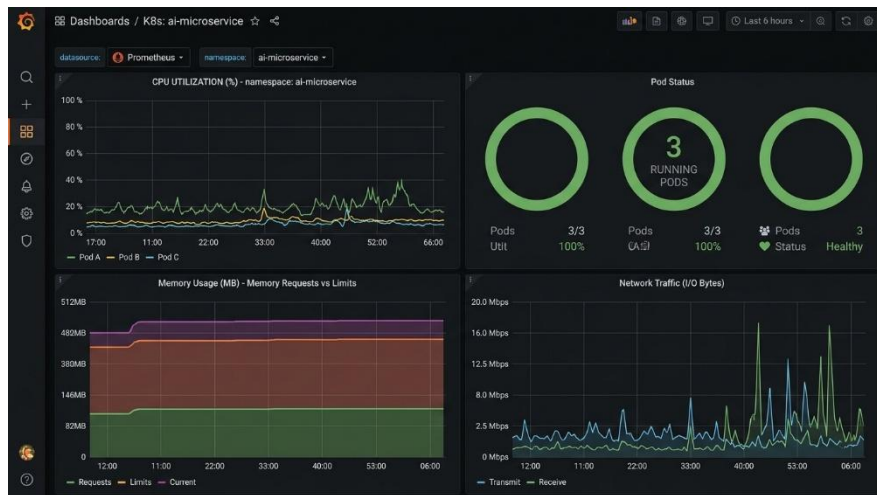


Figure 6.1: Live Grafana EKS Pod Metrics Monitor

## 6.2 Loki Logs Explorer

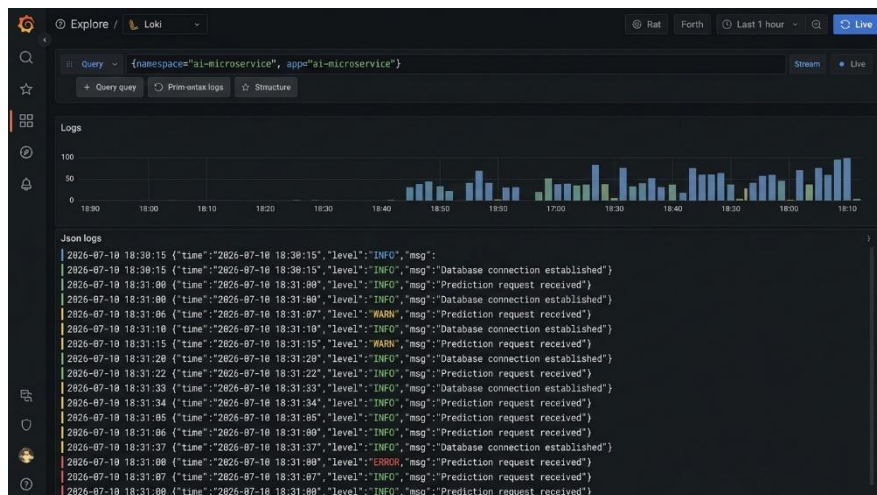


Figure 6.2: Centralized Logging Stream logs in Grafana Explore